

COMS/SE 4170 Software Testing Spring 2026

Control Flow Graphs

1

Overview

- A common **application of graph criteria** is to **program source code**
- Graph : Usually the **control flow graph** (CFG)
- **Node coverage**: Execute every statement
- **Edge coverage**: Execute every branch
- **Loops**: Looping structures such as for loops, while loops, etc.
- **Data flow coverage**: Augment the CFG
 - **defs** are statements that assign values to variables
 - **uses** are statements that use variables

Introduction to Software Testing, Edition 2 (C3.1)

© Ammann & Offutt

2

Control Flow Graphs

- A **CFG** models all executions of a method by describing control structures
- **Nodes**: Statements or sequences of statements (basic blocks)
- **Edges**: Transfers of control
- **Basic Block**: A sequence of statements such that if the first statement is executed, all statements will be (no branches)

Introduction to Software Testing, Edition 2 (C3.1)

© Ammann & Offutt

3

Control Flow Graphs

- CFGs are sometimes **annotated** with extra information
 - branch predicates
 - defs
 - uses
- Rules for translating statements into graphs ...

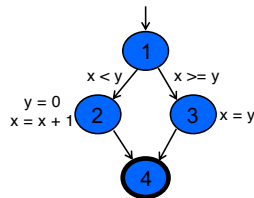
Introduction to Software Testing, Edition 2 (C3.1)

© Ammann & Offutt

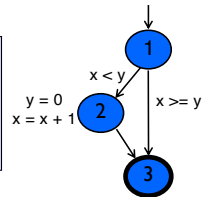
4

CFG : The if Statement

```
if (x < y)
{
  y = 0;
  x = x + 1;
}
else
{
  x = y;
}
```



```
if (x < y)
{
  y = 0;
  x = x + 1;
}
```



Introduction to Software Testing, Edition 2 (Ch 7)

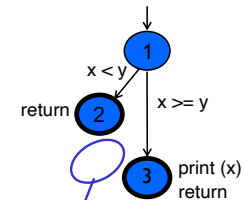
© Ammann & Offutt

5

5

CFG : The if-Return Statement

```
if (x < y)
{
  return;
}
print (x);
return;
```



No edge from node 2 to 3.
The return nodes must be distinct.

Introduction to Software Testing, Edition 2 (Ch 7)

© Ammann & Offutt

6

6

Loops

- Loops require “extra” nodes to be added
- Nodes that do not represent statements or basic blocks

Introduction to Software Testing, Edition 2 (Ch 7)

© Ammann & Offutt

7

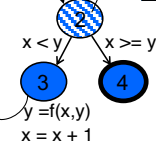
7

CFG : while and for Loops

```
x = 0;
while (x < y)
{
  y = f(x, y);
  x = x + 1;
}
return (x);
```

Draw the graph
and label the
edges.

dummy node



implicitly
initializes loop

Draw the graph
and label the
edges.

```
for (x = 0; x < y; x++)
{
  y = f(x, y);
}
return (x);
```

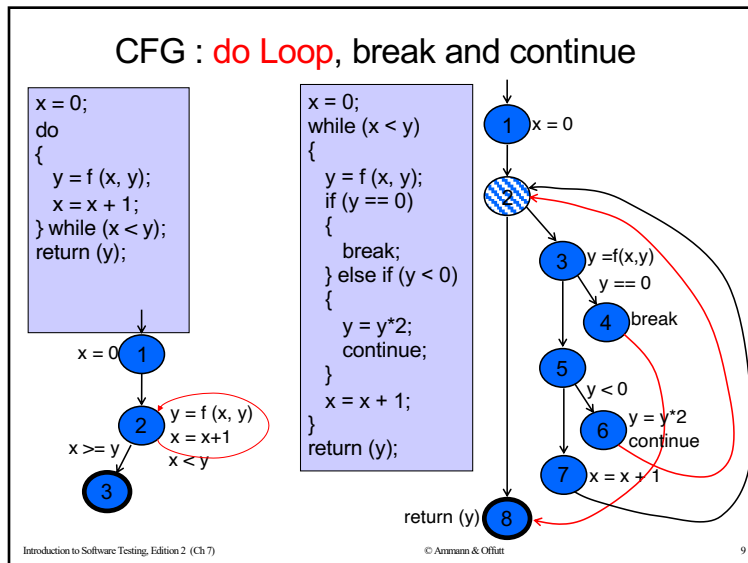
implicitly
increments loop

Introduction to Software Testing, Edition 2 (Ch 7)

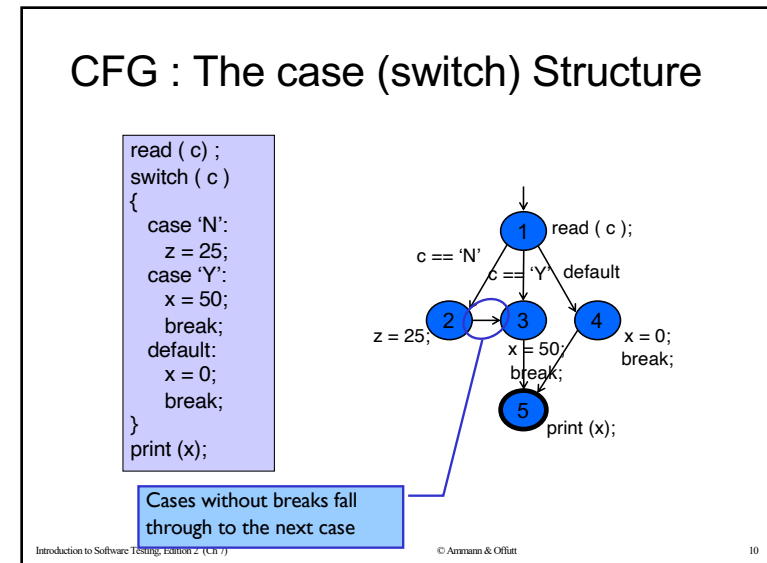
© Ammann & Offutt

8

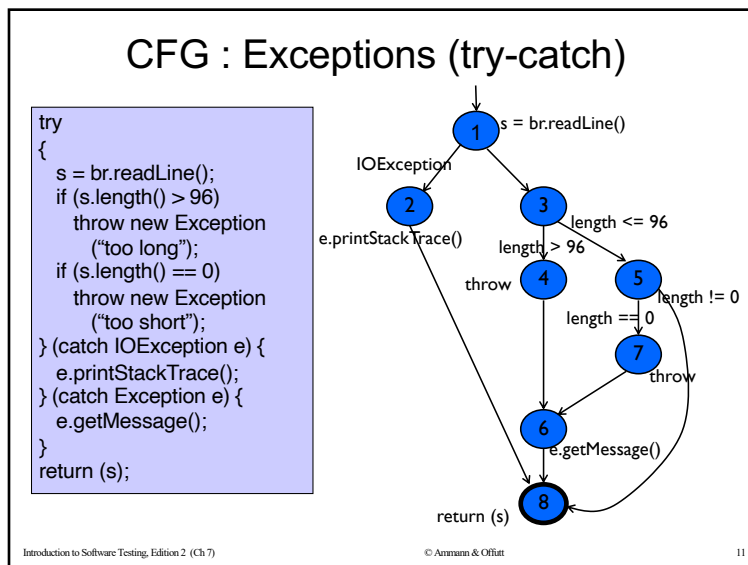
8



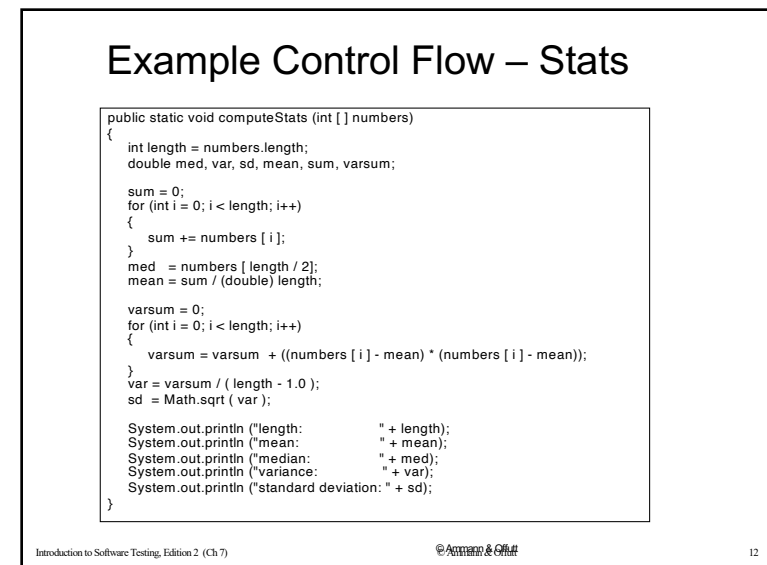
9



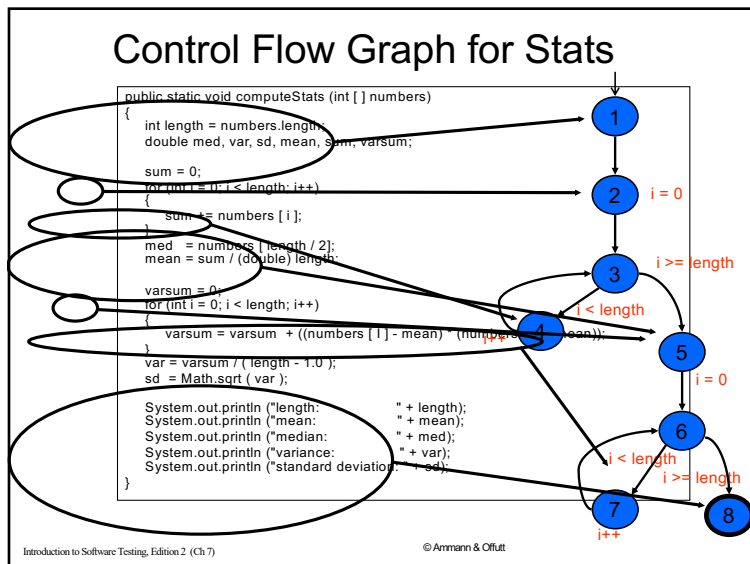
10



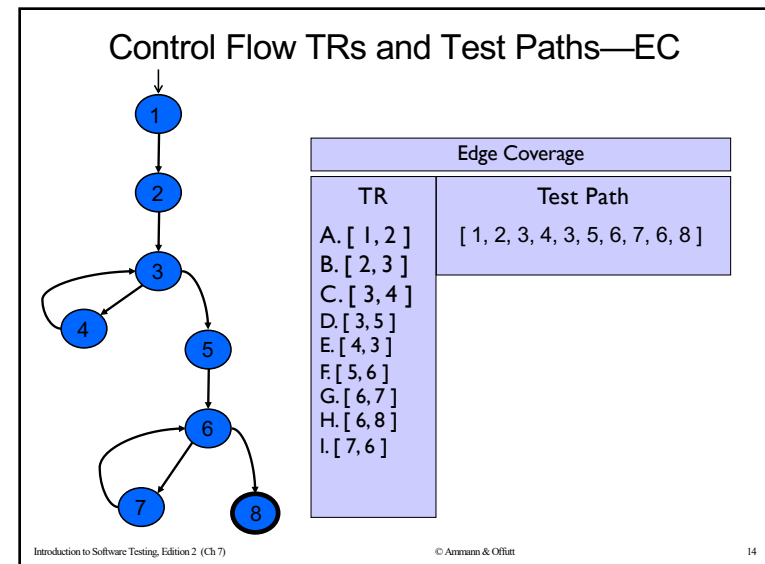
11



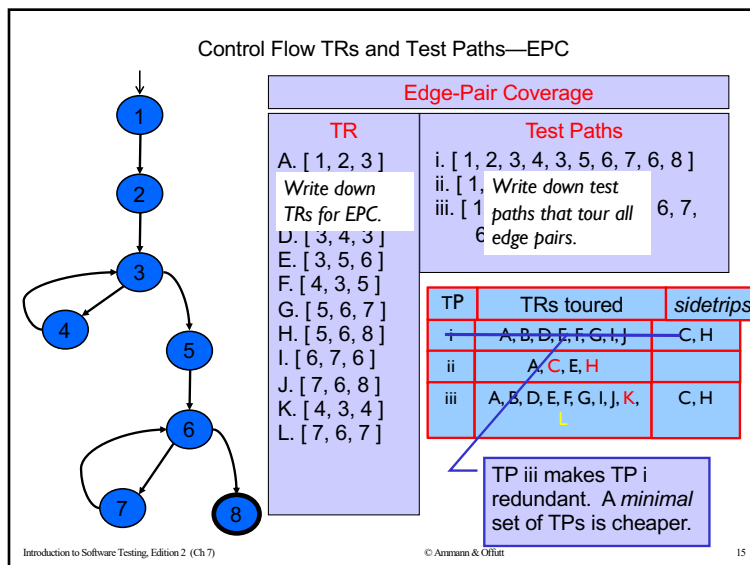
12



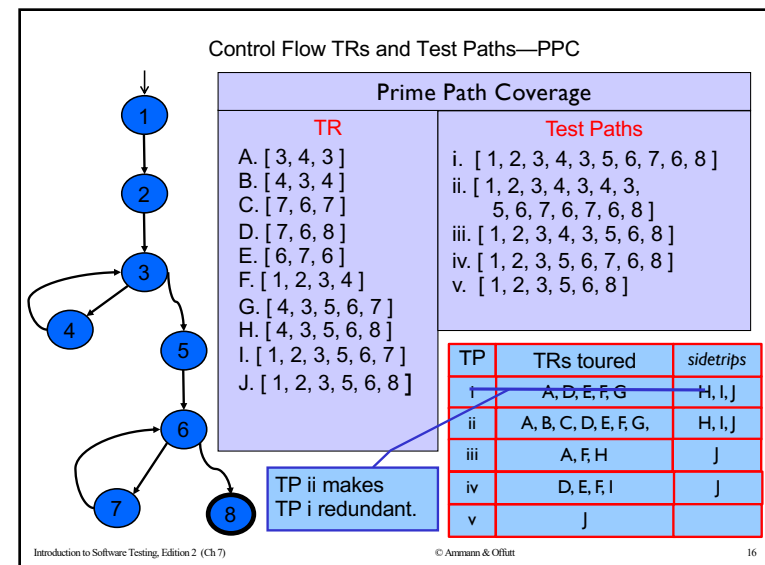
13



14



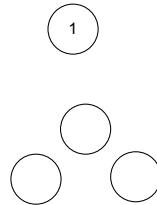
15



16

Another Example

```
x := 0;
y := 0;
while (n > 0) {
  if (n % 2 = 0) {
    x := x + n;
    y := y + 1;
  }
  else {
    y := y + n;
    x := x + 1;
  }
  n := n - 1;
}
print(x);
```



17

Data Flow Coverage for Source

- **def** : a location where a value is stored into memory
 - x appears on the left side of an assignment (x = 44;)
 - x is an actual parameter in a call and the method changes its value
 - x is a formal parameter of a method (implicit def when method starts)
 - x is an input to a program

Introduction to Software Testing, Edition 2 (Ch 7)

© Ammann & Offutt

18

18

Data Flow Coverage for Source

- **use** : a location where variable's value is accessed
 - x appears on the right side of an assignment
 - x appears in a conditional test
 - x is an actual parameter to a method
 - x is an output of the program
 - x is an output of a method in a return statement
- If a **def** and a **use** appear on the same node, then it is only a DU-pair if the def occurs after the use and the node is in a loop

Introduction to Software Testing, Edition 2 (Ch 7)

© Ammann & Offutt

19

19

```
public static void computeStats (int [ ] numbers)
{
  int length = numbers.length;
  double med, var, sd, mean, sum, varsum;

  sum = 0.0;
  for (int i = 0; i < length; i++)
  {
    sum += numbers [ i ];
  }
  med = numbers [ length / 2 ];
  mean = sum / (double) length;

  varsum = 0.0;
  for (int i = 0; i < length; i++)
  {
    varsum = varsum + ((numbers [ i ] - mean) * (numbers [ i ] - mean));
  }
  var = varsum / ( length - 1 );
  sd = Math.sqrt ( var );

  System.out.println ("length:      " + length);
  System.out.println ("mean:      " + mean);
  System.out.println ("median:    " + med);
  System.out.println ("variance:  " + var);
  System.out.println ("standard deviation: " + sd);
}
```

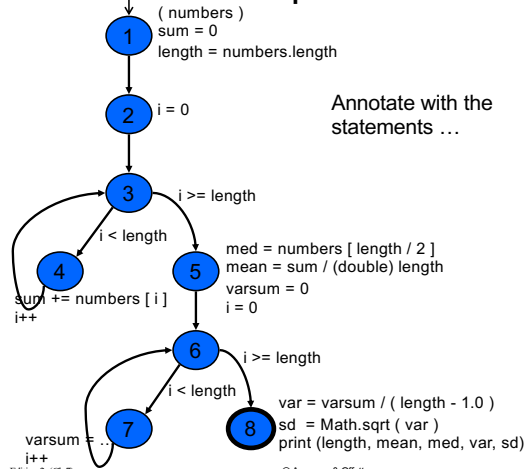
Introduction to Software Testing, Edition 2 (Ch 7)

© Ammann & Offutt

20

20

Control Flow Graph for Stats



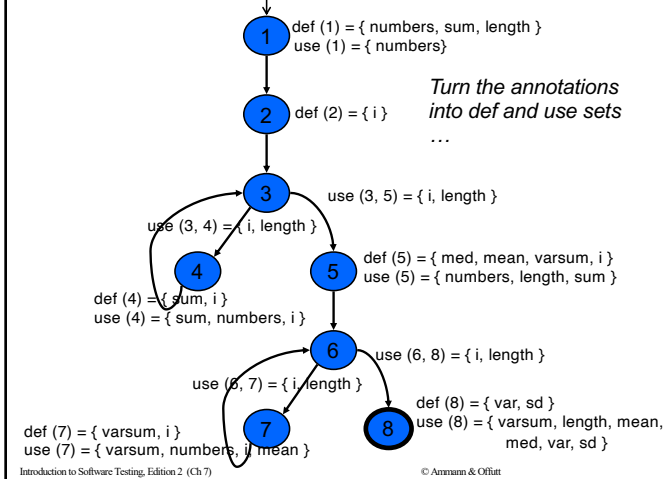
Introduction to Software Testing, Edition 2 (Ch 7)

© Ammann & Offutt

21

21

CFG for Stats – With Defs & Uses



Introduction to Software Testing, Edition 2 (Ch 7)

© Ammann & Offutt

22

22

Defs and Uses Tables for Stats

Node	Def	Use	Edge	Use
1	{ numbers, sum, length }	{ numbers }	(1, 2)	
2	{ i }		(2, 3)	
3			(3, 4)	{ i, length }
4	{ sum, i }	{ numbers, i, sum }	(4, 3)	
5	{ med, mean, varsum, i }	{ numbers, length, sum }	(3, 5)	{ i, length }
6			(5, 6)	
7	{ varsum, i }	{ varsum, numbers, i, mean }	(6, 7)	{ i, length }
8	{ var, sd }	{ varsum, length, var, mean, med, var, sd }	(7, 6)	
			(6, 8)	{ i, length }

Introduction to Software Testing, Edition 2 (Ch 7)

© Ammann & Offutt

23

23

Defs and Uses Tables for Stats

Node	Def	Use	Edge	Use
<pre> public static void computeStats (int [] numbers) { int length = numbers.length; double med, var, sd, mean, sum, varsum; sum = 0.0; for (int i = 0; i < length; i++) { sum += numbers [i]; } med = numbers [length / 2]; mean = sum / (double) length; varsum = 0.0; for (int i = 0; i < length; i++) { varsum = varsum + ((numbers [i] - mean) * (numbers [i] - mean)); } var = varsum / (length - 1); sd = Math.sqrt (var); System.out.println ("length: " + length); System.out.println ("mean: " + mean); System.out.println ("median: " + med); System.out.println ("variance: " + var); System.out.println ("standard deviation: " + sd); } </pre>				
				{ i, length }
				{ i, length }
				{ i, length }
				{ i, length }
				{ i, length }

24

24

DU Pairs for Stats

variable	DU Pairs	defs come <u>before</u> uses, do not count as DU pairs These are local uses
numbers	(1, 4) (1, 5) (1, 7)	
length	(1, 5) (1, 8) (1, (3,4)) (1, (3,5)) (1, (6,7)) (1, (6,8))	
med	(5, 8)	
var	(8, 8)	defs <u>after</u> use in loop, these are valid DU pairs
sd	(8, 8)	
mean	(5, 7) (5, 8)	
sum	(1, 4) (1, 5) (4, 4) (4, 5)	No def-clear path ... different scope for i
varsum	(5, 7) (5, 8) (7, 7) (7, 8)	
i	(2, 4) (2, (3,4)) (2, (3,5)) (2, 7) (2, (6,7)) (2, (6,8)) (4, 4) (4, (3,4)) (4, (3,5)) (4, 7) (4, (6,7)) (4, (6,8)) (5, 7) (5, (6,7)) (5, (6,8)) (7, 7) (7, (6,7)) (7, (6,8))	No path through graph from nodes 5 and 7 to 4 or 3

Introduction to Software Testing, Edition 2 (Ch 7)

© Ammann & Offutt

25

25

DU Paths for Stats

variable	DU Pairs	DU Paths	variable	DU Pairs	DU Paths
numbers	(1, 4) (1, 5) (1, 7)	[1, 2, 3, 4] [1, 2, 3, 5] [1, 2, 3, 5, 6, 7]	mean	(5, 7) (5, 8)	[5, 6, 7] [5, 6, 8]
length	(1, 5) (1, 8) (1, (3,4)) (1, (3,5)) (1, (6,7)) (1, (6,8))	[1, 2, 3, 5] [1, 2, 3, 5, 6, 8] [1, 2, 3, 4] [1, 2, 3, 5] [1, 2, 3, 5, 6, 7] [1, 2, 3, 5, 6, 8]	varsum	(5, 7) (5, 8) (7, 7) (7, 8)	[5, 6, 7] [5, 6, 8] [7, 6, 7] [7, 6, 8]
med	(5, 8)	[5, 6, 8]	i	(2, 4) (2, (3,4)) (2, (3,5)) (4, 4) (4, (3,4)) (4, (3,5)) (5, 7) (5, (6,7)) (5, (6,8)) (7, 7) (7, (6,7)) (7, (6,8))	[2, 3, 4] [2, 3, 4] [2, 3, 5] [4, 3, 4] [4, 3, 4] [4, 3, 5] [5, 6, 7] [5, 6, 7] [5, 6, 8] [7, 6, 7] [7, 6, 7] [7, 6, 8]
var	(8, 8)	No path needed			
sd	(8, 8)	No path needed			
sum	(1, 4) (1, 5) (4, 4) (4, 5)	[1, 2, 3, 4] [1, 2, 3, 5] [4, 3, 4] [4, 3, 5]			

Introduction to Software Testing, Edition 2 (Ch 7)

© Ammann & Offutt

26

26

DU Paths for Stats—No Duplicates

There are 38 DU paths for Stats, but only 12 unique

★ [1, 2, 3, 4]	⚙ [4, 3, 4]
★ [1, 2, 3, 5]	⚙ [4, 3, 5]
★ [1, 2, 3, 5, 6, 7]	⚙ [5, 6, 7]
★ [1, 2, 3, 5, 6, 8]	⚙ [5, 6, 8]
★ [2, 3, 4]	⚙ [7, 6, 7]
★ [2, 3, 5]	⚙ [7, 6, 8]
★	4 expect a loop not to be "entered"
★	6 require at least one iteration of a loop
⚙	2 require at least <u>two</u> iterations of a loop

Introduction to Software Testing, Edition 2 (Ch 7)

© Ammann & Offutt

27

27

Test Cases and Test Paths

Test Case : numbers = (44) ; length = 1
 Test Path : [1, 2, 3, 4, 3, 5, 6, 7, 6, 8]
 Additional DU Paths covered (no sidetrips)
 [1, 2, 3, 4] [2, 3, 4] [4, 3, 5] [5, 6, 7] [7, 6, 8]
 The five stars★ that require at least one iteration of a

Test Case : numbers = (2, 10, 15) ; length = 3
 Test Path : [1, 2, 3, 4, 3, 4, 3, 4, 3, 5, 6, 7, 6, 7, 6, 7, 6, 8]
 DU Paths covered (no sidetrips)
 [4, 3, 4] [7, 6, 7]
 The two stars ⚙ that require at least two iterations of a

Other DU paths ★ require arrays with length 0 to skip loops
 But the method fails with index out of bounds exception...
 med = numbers [length / 2];

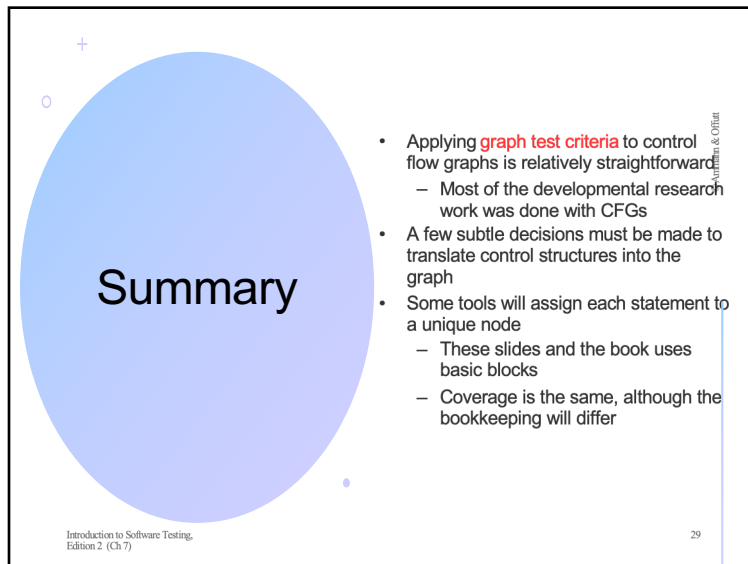
A fault was found

Introduction to Software Testing, Edition 2 (Ch 7)

© Ammann & Offutt

28

28



Summary

- Applying **graph test criteria** to control flow graphs is relatively straightforward
 - Most of the developmental research work was done with CFGs
- A few subtle decisions must be made to translate control structures into the graph
- Some tools will assign each statement to a unique node
 - These slides and the book uses basic blocks
 - Coverage is the same, although the bookkeeping will differ

Introduction to Software Testing,
Edition 2 (Ch 7)

29